

# ProjWT — WHOOP 4.0 Reverse-Engineering Build Log

**Project:** ProjWT (personal portfolio project — private) **Owner:** Hamza Khan **Status:** Phase 1–2 (BLE access + protocol) — in progress **Last updated:** 2026-10-01

**PRIVATE / PORTFOLIO.** This document and the whole ProjWT repo are a personal portfolio piece and are not to be shared publicly.

This is the **living record** of the project — every step, command, result, and decision. It is the source of truth; the PDF/Word versions are generated from this file (see *\*Regenerating the PDF/Word\** at the end). Keep it updated as we go.

---

## 1. What we're building (and the safety line)

A personal system that talks directly to a WHOOP 4.0 band over Bluetooth Low Energy (BLE), reads its sensors, and eventually drives:

- **Biometric dashboard** — live heart rate, HRV, SpO<sub>2</sub>, skin temp, motion.
- **Gesture control** — recognise wrist gestures from the band's accelerometer.
- **Vehicle control (Tesla Fleet API)** — \*supervised only.\*
- **Home / IoT automation.**

**Target stack (eventual):** C++ / Unreal Engine 5. **Phase-1 tooling (now):** Python — fastest way to reverse-engineer and validate the protocol before committing it to C++.

### ***Safety constraint (non-negotiable)***

Vehicle control is **active-hold, line-of-sight only**:

- You hold a gesture **while watching the car**; the car moves only while the gesture is actively held.
- Releasing the gesture (a dead-man's-switch) slows or stops the car.
- **No** "no-line-of-sight summon", **no** GPS spoofing, **no** safety-bypass.

That idea was considered and **dropped**.

---

## 2. The hardware target

- **Device:** WHOOP 4.0 band (owned by the project author).
- **Advertised name:** WHOOP\_4C1656042
- **BLE address (Windows, this session):** D1:86:73:D4:62:86
- ■■ On Windows this address can be a rotating/resolvable random address, so **re-scan each session** rather than hard-coding it (see §8).

- **Battery at bring-up:** 100% (band healthy, charged via the pack; tap → LED).

---

### 3. Environment setup (Windows)

The band talks to the PC directly over BLE — **no WHOOP app, no pairing needed**. We drive it from Python.

- 1 **Install Python** (3.10+). During install tick **"Add Python to PATH."**

- 2 **Install the BLE library:**

```
python -m pip install bleak
```

- 1 **Open a terminal in the code folder.** In File Explorer, open the projwt\src folder, click the address bar, type powershell, press Enter.

- 1 **Run scripts with python first.** PowerShell will **\*not\*** run ble\_scanner\_PWT.py on its own (that just opens Notepad). Always:

```
python ble_scanner_PWT.py --name whoop
```

**Common beginner errors we already hit and fixed** - **"The term 'ble\_scanner\_PWT.py' is not recognized"** → you forgot the python prefix. Start the command with python. - **"Pressing Enter opens Notepad"** → same cause; Windows is "opening" the file instead of running it. Use python <file>. - **"ModuleNotFoundError: No module named 'bleak'"** → run python -m pip install bleak.

---

### 4. The tools we built (in src/, all \_PWT)

| Script                | Purpose                                                                                |
|-----------------------|----------------------------------------------------------------------------------------|
| ble_scanner_PWT.py    | Discover BLE devices, print name / address / RSSI. Find the band.                      |
| gatt_explorer_PWT.py  | Enumerate the band's GATT services / characteristics / descriptors.                    |
| ble_logger_PWT.py     | Subscribe to notify channels and log raw packets to logs/ (JSONL). --label, --seconds. |
| whoop_protocol_PWT.py | <b>The verified protocol core</b> — framing, CRCs, command IDs, UUIDs.                 |
| ble_command_PWT.py    | Send one framed command and listen for the reply.                                      |
| ble_stream_PWT.py     | <b>HELLO</b> → <b>START</b> → <b>watch data channel</b> for a continuous stream.       |
| ble_sweep_PWT.py      | Sweep a range of command IDs to find which one wakes the data channel.                 |

| Script                       | Purpose                                                                  |
|------------------------------|--------------------------------------------------------------------------|
| <code>ble_pair_PWT.py</code> | Pair/bond with the band so it will accept our commands (encrypted link). |

Everything is committed to the private repo Hamza-Khan-algonquin/projw, branch `claude/peaceful-dirac-ko6o1i`.

---

## 5. What we did, step by step (chronological)

- 1 **Charged and woke the band.** Tapping it showed the LED; battery reached 100%. Windows Bluetooth couldn't see it at first (it wasn't advertising / the WHOOP app's own pairing kept crashing). We bypassed all of that and scanned directly from Python.
- 1 **Scanned with `ble_scanner_PWT.py`.** Found WHOOP 4C1656042 at D1:86:73:D4:62:86, RSSI  $\approx -61$  dBm (strong, close).
- 1 **Mapped the GATT table with `gatt_explorer_PWT.py`.** Got the custom WHOOP service and its characteristics (see §6).
- 1 **Logged raw notifications with `ble_logger_PWT.py`.** Got **0 packets** at rest — *\*expected\**: the band stays silent until told to stream. Also fixed the ugly Ctrl+C traceback and added `--seconds` auto-stop.
- 1 **Reverse-engineered the packet framing.** Community write-ups disagreed with each other (they were paraphrased summaries), so we **brute-forced the CRC parameters against a real captured packet inside a sandbox** until the framing reproduced byte-for-byte. This is now the verified core (§6).
- 1 **Sent our first real command with `ble_command_PWT.py`.** The band *\*\*replied* with a valid, CRC-correct packet — **proof our framing is accepted by real hardware (two-way comms confirmed)**. The reply came on the *event\*\** channel (61080003), 32 bytes, a protobuf-style status/report — *\*not\** a continuous sensor stream.
- 1 **Diagnosed why there was no stream.** Reference implementations perform a **HELLO handshake (cmd 0x05) first**, then START; the continuous data arrives as ~96-byte packets on the **data** channel (61080004). We had skipped the hello. Built `ble_stream_PWT.py` to do the full sequence.
- 1 **Ran the full HELLO → START sequence (`ble_stream_PWT.py`).** The band came alive — **15 packets in 30s** — but **all on the EVENT channel (61080003)**, a repeating ~2s **status beacon**, *\*not\** the 96-byte sensor stream on the DATA channel (61080004, which stayed silent). Two outcomes:
  - **Decoded the beacon's identity fields** (see §6.5): codename `boylston`, firmware **17.2.2.0**, hardware `harvard_r10`.
  - **Found & fixed a display bug:** the tools labelled channels by the *\*last\** 8

UUID chars, but every WHOOP characteristic ends in ...b0f8dcc6, so all channels printed the same label. They differ in the \*first\* block (6108000x); labelling now uses a clear name map (EVENT(03), DATA(04)...).

## 1 Concluded cmd 0x03 is not the real "start streaming" opcode (the command

IDs were always community guesses). Built `ble_sweep_PWT.py` to brute-force the correct opcode — the same empirical approach that cracked the CRC framing.

## 1 Swept command IDs (`ble_sweep_PWT.py`) — and learned two big things:

### ● The band ignores unauthenticated commands. Write-\*with\*-response returned

GATT Protocol Error: Insufficient Authentication (ATT error 0x05 — the command characteristic needs an **encrypted / paired link**). Write-\*without\*-response (our default) doesn't error, but the band **silently ignores it**. So \*none of our commands had ever actually reached the band\* — every "reply" so far was the subscribe-triggered status beacon or physical motion.

### ● DATA(04) is currently a motion/tap event feed. The DATA-channel packets

lined up with \*tapping / shaking / putting the charger on\*, not with any command. They use our verified 0xAA framing, **type** 0x30, with an incrementing sequence and a monotonic **timestamp** — i.e. timestamped accelerometer/activity events (useful later for gesture detection).

## 1 Built `ble_pair_PWT.py` and added `--pair` to the sweep/stream tools.

Pairing establishes the encrypted link so the band will honour commands.

## 1 ← You are here. Next: **pair**, then re-sweep **with the band held still**

so any DATA response is attributable to the command (not motion), find the real start opcode, then build the decoder.

---

## 6. The verified protocol (the important part)

### 6.1 GATT map (from our own scan — authoritative)

- **Service:** 61080001-8d6d-82b8-614a-1c8cb0f8dcc6
- **Command (write):** 61080002-... — where we send commands.
- **Event / replies (notify):** 61080003-... — status/report packets.
- **Data (notify):** 61080004-... — real-time sensor packets (~96 bytes).
- **Diagnostics (notify):** 61080005-...
- **Extra (notify):** 61080007-...
- **Standard Heart Rate (notify):** 00002a37-0000-1000-8000-00805f9b34fb

Some public repos label these UUIDs off-by-one. **Our scan wins.**

### 6.2 Packet framing — empirically verified

0xAA | length(2, little-endian) | CRC8(length) | PAYLOAD | CRC32(PAYLOAD, LE)

length = len(PAYLOAD) + 4 (payload plus the trailing CRC32)  
CRC8 = poly 0x07, init 0x00, no reflection, over the 2 length bytes

```
CRC32    = standard zlib.crc32 over PAYLOAD, stored little-endian
PAYLOAD  = [type][seq][cmd][data...]
type     = 0x23 for commands
```

**Proof:** `build_packet()` reproduces the known-good reference packet exactly:

```
aa100057230423aa8ed469a96d0000005130fef3  ← reference
aa100057230423aa8ed469a96d0000005130fef3  ← our build_packet()  ✓ MATCH
```

Run the self-test any time: `python whoop_protocol_PWT.py`.

■■ The MIT reference repo uses a *\*different, simpler\** framing (no CRC8). We do **not** use it — **our** framing is the one the real band actually replied to.

### 6.3 Command IDs (community-sourced; being verified live)

| Cmd  | Meaning                               |
|------|---------------------------------------|
| 0x01 | Get battery                           |
| 0x02 | Get device info (fw / serial / hw)    |
| 0x03 | Start real-time streaming             |
| 0x04 | Stop real-time streaming              |
| 0x05 | <b>HELLO / handshake — send first</b> |

### 6.4 Real-time data packet (~96 bytes, on 61080004) — *\*tentative layout\**

Adapted from the reference parser; **byte offsets past temperature are guesses** and will be validated against our own captures before we trust them.

| Bytes                | Field (tentative)                  |
|----------------------|------------------------------------|
| [0]                  | sequence                           |
| [1:3]                | heart rate × 100 (uint16 LE) → BPM |
| [3:5]                | RR interval, ms (uint16 LE)        |
| [5]                  | SpO■■                              |
| [6]                  | skin temperature °C (offset +25)   |
| [7:9] [9:11] [11:13] | accelerometer X / Y / Z (int16)    |
| [13]                 | motion flag                        |
| [14:16]              | PPG amplitude                      |
| [16:18]              | ambient light                      |
| [18:20]              | PPG quality                        |
| [20:91]              | unknown / reserved                 |
| [92:96]              | CRC32 (LE)                         |

Captured **event** packet (32 bytes on 61080003, protobuf body) for reference:

```
aa 1c 00 ab 30 06 1d 00 2d 48 e1 01 20 1a 0c 00 32 01 00 00 02 00 42 02 0b 00 01 00 8c 60 8f ce
```

## 6.5 Status beacon (EVENT channel 61080003) — decoded identity

When we subscribe, the band emits a ~97-byte **status beacon every ~2 seconds** on the EVENT channel. It is a protobuf body (starts 08 02 a6 02 ...) carrying the device identity plus a changing counter near the tail. Decoded ASCII fields:

| Field              | Value       |
|--------------------|-------------|
| Codename           | boylston    |
| Firmware / version | 17.2.2.0    |
| Hardware revision  | harvard_r10 |

This is a **heartbeat/status** feed, not the per-sample sensor stream. The per-sample sensor data (PPG / accel / HR) is expected as ~96-byte packets on the **DATA channel** 61080004, which is still silent — see §9 next steps.

## 6.6 Authentication requirement (the key blocker)

The command characteristic (61080002) requires an **encrypted/paired BLE link**. Evidence: write-with-response returns GATT Protocol Error: Insufficient Authentication (ATT error 0x05). Write-without-response doesn't raise, but the band silently discards the write. **Therefore commands only take effect after we pair/bond** (ble\_pair\_PWT.py, or --pair). Until then the band only emits the status beacon and motion events, regardless of what we send.

## 6.7 DATA(04) event packets (motion-triggered) — framing confirmed

The DATA-channel packets observed so far are our 0xAA frame with **type** 0x30:

```
aa | len(2 LE) | crc8(len) | 30 | seq | .. | timestamp(uint32 LE) | .. | crc32
```

Example: aa 10 00 57 30 24 0e 00 c8 87 e2 01 80 44 00 00 ... → len=16, crc8 OK, type=0x30, seq=0x24, timestamp=0x01e287c8 (monotonic). These fire on tap/shake/charger — accelerometer/activity events, not the continuous PPG stream we still need to unlock (via pairing + the right opcode).

---

## 7. Legal / attribution — do the referenced authors need to know?

**Short answer: no obligation to notify anyone, and you don't have to publish that you used their work.**

- The reference we consulted, christianmeurer/whoop-reader, is MIT-licensed.

The MIT license lets you use, copy, modify, and even redistribute the code — including in a closed/private project — **with one condition**: \*if you redistribute their source or substantial portions of it, you must keep their copyright notice + the MIT license text with it.\*

- **You are not "redistributing" it.** We only **read** it for reference and wrote

our **own** implementation (in fact with \*different, independently verified\* framing). So the MIT condition isn't even triggered.

- **The author is never automatically notified.** GitHub doesn't tell someone when

you read, clone, or reference their repo. There's no tracking, no callback.

- **Do you \*have\* to "let it be known"?** No. But \*\*crediting them is good

practice\*\* and costs nothing — so we've added a short credit note in `whoop_protocol_PWT.py` and here. It also makes the portfolio piece look more professional (shows you research prior art and respect licenses).

- The other names in the space (jogolden/whoomp, bwanShiTong/reverse-engineering-whoop) — same logic. jogolden/whoomp couldn't be added to this session (not found/accessible), so we used the MIT whoop-reader instead.

**Bottom line:** keep the credit note (already added), don't paste their source verbatim into your repo, and you're completely fine — legally and ethically.

---

## 8. Reconnecting after a break (start-of-session checklist)

The band is stateless from our side — nothing persists on the PC. Next time you sit down:

- 1 **Charge the band** if it's been idle a while; tap it to confirm the LED.
- 2 **Open PowerShell in the code folder** — File Explorer → projwt\src folder  
→ type `powershell` in the address bar → Enter.

- 1 **Re-scan for the band** (the address may have changed):

```
python ble_scanner_PWT.py --name whoop
```

Copy the address it prints (e.g. D1:86:73:D4:62:86).

- 1 **(Optional) confirm the GATT map is unchanged:**

```
python gatt_explorer_PWT.py <ADDRESS>
```

- 1 **Pair once** (only needed the first time, or after `--unpair`; the bond persists across sessions):

```
python ble_pair_PWT.py <ADDRESS>
```

- 1 **Find the stream opcode** (current task — see §9), band held still:

```
python ble_sweep_PWT.py <ADDRESS> --pair --with-response
```

Once we know the right opcode, stream with:

```
python ble_stream_PWT.py <ADDRESS> --pair --with-response --seconds 30
```

- 1 **Send Claude the newest file in logs\** so we can decode/continue.

If bleak is missing again (new machine): `python -m pip install bleak`. If nothing is found in the scan: make sure the band is charged and near the PC, and that no other app (the WHOOP phone app) is holding the connection.

---

## 9. Current status & next steps

**Done**

- Physical bring-up, BLE discovery, full GATT map.

- Packet framing reverse-engineered **and verified both ways** (we sent a command and the real band returned a valid CRC-checked reply).
- Confirmed two-way comms live; decoded the device-identity status beacon (firmware 17.2.2.0, hardware harvard\_r10).
- Tooling for scan / explore / log / command / stream / sweep.

## Next

### 1 Pair with the band (one time):

```
python ble_pair_PWT.py <ADDRESS>
```

Accept any Windows pairing prompt. "already paired" is fine.

### 1 Re-sweep over the encrypted link, band held perfectly still (so any DATA response is from the command, not motion):

```
python ble_sweep_PWT.py <ADDRESS> --pair --with-response
```

Watch for the command id whose DATA(04) response \*continues after the dwell\* (a real stream), vs. a one-off reply. Send Claude the `logs\sweep_*.jsonl`.

### 1 Lock the start opcode into ble\_stream\_PWT.py and capture a real sensor stream (wear the band, tight skin contact):

```
python ble_stream_PWT.py <ADDRESS> --pair --with-response --seconds 30
```

### 1 Build decode\_realtime\_PWT.py from the captured bytes — validate HR against your actual resting heart rate, then lock in the accel / temp / SpO<sub>2</sub> offsets.

### 1 Gesture recognition from the accelerometer stream.

### 2 Port the verified protocol to C++ for the UE5 phase.

---

## 10. Regenerating the PDF / Word versions

This Markdown file is the master. The PDF and Word copies are generated from it with a small pure-Python script (no LibreOffice needed, so it runs on your Windows machine too). One-time setup:

```
python -m pip install reportlab python-docx
```

Then, from the repo root, regenerate `BUILD_LOG_PWT.pdf` and `BUILD_LOG_PWT.docx` any time you edit this file:

```
python Tools/export_docs_PWT.py
```

Re-run it whenever you update this log so the shareable copies stay in sync.

---

\*ProjWT build log — private portfolio project. Keep this file updated.\*